

1. 프로젝트 환경설정

#0. 강의/4.jpa 활용로드맵/datajpa/강의

- /프로젝트 생성
- /라이브러리 살펴보기
- /H2 데이터베이스 설치
- /스프링 데이터 JPA와 DB 설정, 동작확인

프로젝트 생성

- 스프링 부트 스타터(<https://start.spring.io/>)
- Project: **Gradle - Groovy** Project
- 사용 기능: web, jpa, h2, lombok
 - SpringBootVersion: **3.x.x**
 - groupId: study
 - artifactId: data-jpa

주의! - 스프링 부트 3.x 버전 선택 필수

start.spring.io 사이트에서 스프링 부트 2.x에 대한 지원이 종료되어서 더는 선택할 수 없습니다.

이제는 스프링 부트 3.0 이상을 선택해주세요.

스프링 부트 3.0을 선택하게 되면 다음 부분을 꼭 확인해주세요.

- **1. Java 17 이상**을 사용해야 합니다.
- **2. javax 패키지 이름을 jakarta로 변경**해야 합니다.
 - 오라클과 자바 라이선스 문제로 모든 javax 패키지를 jakarta로 변경하기로 했습니다.
- **3. H2 데이터베이스를 2.1.214 버전 이상** 사용해주세요.

패키지 이름 변경 예)

- **JPA 애노테이션**
 - javax.persistence.Entity → jakarta.persistence.Entity
- 스프링에서 자주 사용하는 **@PostConstruct 애노테이션**
 - javax.annotation.PostConstruct → jakarta.annotation.PostConstruct
- 스프링에서 자주 사용하는 **검증 애노테이션**
 - javax.validation → jakarta.validation

스프링 부트 3.0 관련 자세한 내용은 다음 링크를 확인해주세요: <https://bit.ly/springboot3>

Gradle 전체 설정

```
plugins {  
    id 'org.springframework.boot' version '2.2.1.RELEASE'  
    id 'io.spring.dependency-management' version '1.0.8.RELEASE'  
    id 'java'  
}  
  
group = 'study'  
version = '0.0.1-SNAPSHOT'  
sourceCompatibility = '1.8'  
  
configurations {  
    compileOnly {  
        extendsFrom annotationProcessor  
    }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'com.github.gavlyukovskiy:p6spy-spring-boot-starter:1.5.7'  
    compileOnly 'org.projectlombok:lombok'  
    runtimeOnly 'com.h2database:h2'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation('org.springframework.boot:spring-boot-starter-test') {  
        exclude group: 'org.junit.vintage', module: 'junit-vintage-engine'  
    }  
}  
  
test {  
    useJUnitPlatform()  
}
```

필독! 주의!

강의 영상이 JUnit4를 기준으로 하기 때문에 build.gradle에 있는 다음 부분을 꼭 직접 추가해주세요.
해당 부분을 입력하지 않으면 JUnit5로 동작합니다. JUnit5를 잘 알고 선호하시면 입력하지 않아도 됩니다.

```
//JUnit4 추가
testImplementation("org.junit.vintage:junit-vintage-engine") {
    exclude group: "org.hamcrest", module: "hamcrest-core"
}
```

- 동작 확인
 - 기본 테스트 케이스 실행
 - 스프링 부트 메인 실행 후 에러페이지로 간단하게 동작 확인(<http://localhost:8080>)
 - 테스트 컨트롤러를 만들어서 spring web 동작 확인(<http://localhost:8080/hello>)

테스트 컨트롤러

```
package study.datajpa.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @RequestMapping("/hello")
    public String hello() {
        return "hello";
    }
}
```

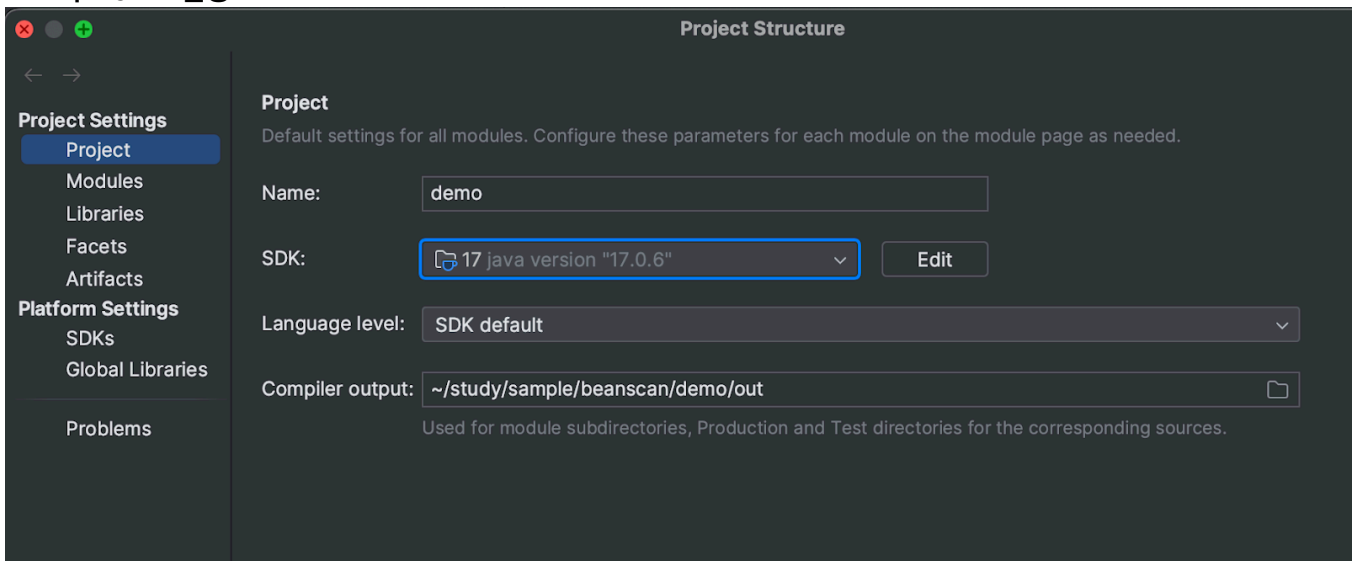
롬복 적용

1. Preferences → plugin → lombok 검색 실행 (재시작)
2. Preferences → Annotation Processors 검색 → Enable annotation processing 체크 (재시작)
3. 임의의 테스트 클래스를 만들고 @Getter, @Setter 확인

주의! 스프링 부트 3.2부터 JDK 17과 Gradle 옵션을 선택하자.

스프링 부트 3.2 부터 JDK 17 이상과, 빌드시 IntelliJ IDEA가 아니라 Gradle을 선택해야 한다.

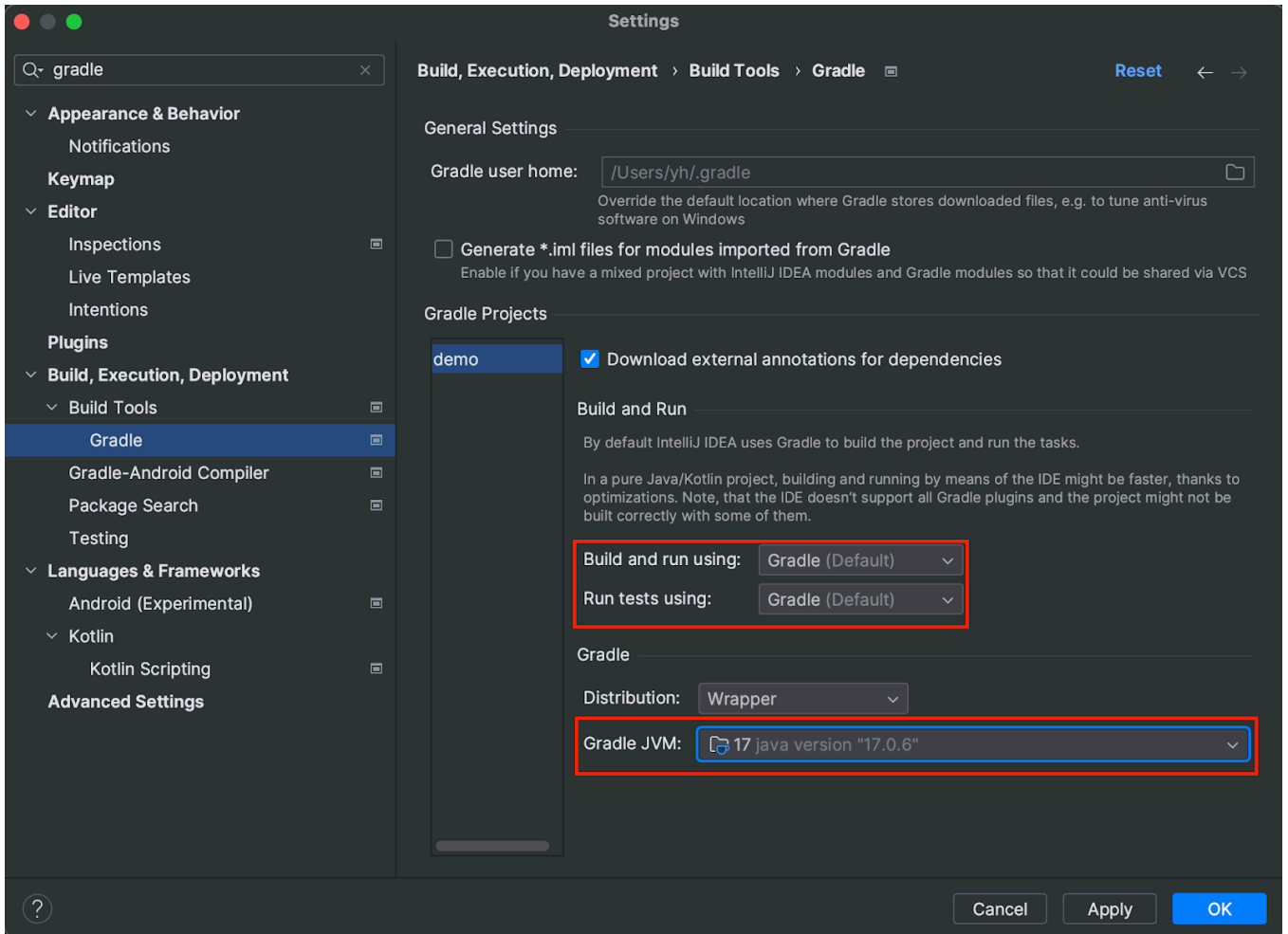
프로젝트 JDK 설정



다음으로 이동합니다.

- Windows: File → Project Structure (Ctrl+Alt+Shift+S)
- Mac: File → Project Structure (⌘;))

파란 박스의 JDK를 내가 새로 설치한 **자바 17로 지정**해줍니다.



다음으로 이동합니다.

- Windows: File → Settings(Ctrl+Alt+S)
- Mac: IntelliJ IDEA | Preferences(⌘,))
- Preferences → Build, Execution, Deployment → Build Tools → Gradle
- 빨간색 박스의 **Build and run using**를 **Gradle**로 선택합니다.
- 빨간색 박스의 **Build tests using**를 **Gradle**로 선택합니다.

빨간색 박스 Gradle JVM을 새로 설치한 자바 17또는 그 이상으로 지정합니다.

라이브러리 살펴보기

gradle 의존관계 보기

```
./gradlew dependencies --configuration compileClasspath
```

스프링 부트 라이브러리 살펴보기

- spring-boot-starter-web
 - spring-boot-starter-tomcat: 톰캣 (웹서버)
 - spring-webmvc: 스프링 웹 MVC
- spring-boot-starter-data-jpa
 - spring-boot-starter-aop
 - spring-boot-starter-jdbc
 - ◆ HikariCP 커넥션 풀 (부트 2.0 기본)
 - hibernate + JPA: 하이버네이트 + JPA
 - spring-data-jpa: 스프링 데이터 JPA
- spring-boot-starter(공통): 스프링 부트 + 스프링 코어 + 로깅
 - spring-boot
 - ◆ spring-core
 - spring-boot-starter-logging
 - ◆ logback, slf4j

테스트 라이브러리

- spring-boot-starter-test
 - junit: 테스트 프레임워크, 스프링 부트 2.2부터 junit5(jupiter) 사용
 - ◆ 과거 버전은 vintage
 - mockito: 목 라이브러리
 - assertj: 테스트 코드를 좀 더 편하게 작성하게 도와주는 라이브러리
 - ◆ <https://joel-costigliola.github.io/assertj/index.html>
 - spring-test: 스프링 통합 테스트 지원
- 핵심 라이브러리
 - 스프링 MVC
 - 스프링 ORM
 - JPA, 하이버네이트
 - 스프링 데이터 JPA
- 기타 라이브러리
 - H2 데이터베이스 클라이언트
 - 커넥션 풀: 부트 기본은 HikariCP
 - 로깅 SLF4J & LogBack
 - 테스트

H2 데이터베이스 설치

주의! H2 데이터베이스를 2.1.214 버전 이상으로 사용해주세요.

개발이나 테스트 용도로 가볍고 편리한 DB, 웹 화면 제공

- <https://www.h2database.com>
- 다운로드 및 설치
 - 스프링 부트 2.x를 사용하면 **1.4.200 버전**을 다운로드 받으면 된다.
 - 스프링 부트 3.x를 사용하면 **2.1.214 버전 이상** 사용해야 한다.
 - h2 데이터베이스 버전은 스프링 부트 버전에 맞춘다.
- 권한 주기: `chmod 755 h2.sh`
- 데이터베이스 파일 생성 방법
 - `jdbc:h2:~/datajpa` (최소 한번)
 - `~/datajpa.mv.db` 파일 생성 확인
 - 이후 부터는 `jdbc:h2:tcp://localhost/~/datajpa` 이렇게 접속

주의: H2 데이터베이스의 MVCC 옵션은 H2 1.4.198 버전부터 제거되었습니다. 최신 버전에서는 **MVCC 옵션**을 사용하면 오류가 발생합니다.

스프링 데이터 JPA와 DB 설정, 동작확인

application.yml

```
spring:
  datasource:
    url: jdbc:h2:tcp://localhost/~/datajpa
    username: sa
    password:
    driver-class-name: org.h2.Driver

  jpa:
    hibernate:
      ddl-auto: create
    properties:
```

```

hibernate:
#     show_sql: true
    format_sql: true

logging.level:
    org.hibernate.SQL: debug
# org.hibernate.type: trace

```

- spring.jpa.hibernate.ddl-auto: create
 - 이 옵션은 애플리케이션 실행 시점에 테이블을 drop 하고, 다시 생성한다.

참고: 모든 로그 출력은 가급적 로거를 통해 남겨야 한다.

show_sql: 옵션은 System.out 에 하이버네이트 실행 SQL을 남긴다.

org.hibernate.SQL: 옵션은 logger를 통해 하이버네이트 실행 SQL을 남긴다.

실제 동작하는지 확인하기

회원 엔티티

```

@Entity
@Getter @Setter
public class Member {

    @Id @GeneratedValue
    private Long id;
    private String username;
    ...
}

```

회원 JPA 리포지토리

```

@Repository
public class MemberJpaRepository {

    @PersistenceContext
    private EntityManager em;
}

```



```

public Member save(Member member) {
    em.persist(member);
    return member;
}

public Member find(Long id) {
    return em.find(Member.class, id);
}
}

```

JPA 기반 테스트

```

@SpringBootTest
@Transactional
@Rollback(false)
public class MemberJpaRepositoryTest {

    @Autowired
    MemberJpaRepository memberJpaRepository;

    @Test
    public void testMember() {
        Member member = new Member("memberA");
        Member savedMember = memberJpaRepository.save(member);

        Member findMember = memberJpaRepository.find(savedMember.getId());

        assertThat(findMember.getId()).isEqualTo(member.getId());
        assertThat(findMember.getUsername()).isEqualTo(member.getUsername());

        assertThat(findMember).isEqualTo(member); //JPA 엔티티 동일성 보장
    }
}

```

스프링 데이터 JPA 리포지토리

```

public interface MemberRepository extends JpaRepository<Member, Long> {

```

```
}
```

스프링 데이터 JPA 기반 테스트

```
@SpringBootTest
@Transactional
@Rollback(false)
public class MemberRepositoryTest {

    @Autowired
    MemberRepository memberRepository;

    @Test
    public void testMember() {
        Member member = new Member("memberA");
        Member savedMember = memberRepository.save(member);

        Member findMember =
memberRepository.findById(savedMember.getId()).get();

        Assertions.assertThat(findMember.getId()).isEqualTo(member.getId());

        Assertions.assertThat(findMember.getUsername()).isEqualTo(member.getUsername())
    );

        Assertions.assertThat(findMember).isEqualTo(member); //JPA 엔티티 동일성
보장
    }

}
```

- Entity, Repository 동작 확인
- jar 빌드해서 동작 확인

참고: 스프링 부트를 통해 복잡한 설정이 다 자동화 되었다. `persistence.xml` 도 없고, `LocalContainerEntityManagerFactoryBean` 도 없다. 스프링 부트를 통한 추가 설정은 스프링 부트 메뉴얼을 참고하고, 스프링 부트를 사용하지 않고 순수 스프링과 JPA 설정 방법은 자바 ORM 표준 JPA 프로그래밍 책을 참고하자.

쿼리 파라미터 로그 남기기

- 로그에 다음을 추가하기 `org.hibernate.type: SQL` 실행 파라미터를 로그로 남긴다.
- 외부 라이브러리 사용
 - <https://github.com/gavlyukovskiy/spring-boot-data-source-decorator>

스프링 부트를 사용하면 이 라이브러리만 추가하면 된다.

```
implementation 'com.github.gavlyukovskiy:p6spy-spring-boot-starter:1.5.7'
```

참고: 쿼리 파라미터를 로그로 남기는 외부 라이브러리는 시스템 자원을 사용하므로, 개발 단계에서는 편하게 사용해도 된다. 하지만 운영시스템에 적용하려면 꼭 성능테스트를 하고 사용하는 것이 좋다.

쿼리 파라미터 로그 남기기 - 스프링 부트 3.0

스프링 부트 3.0 이상을 사용하면 라이브러리 버전을 1.9.0 이상을 사용해야 한다.

```
implementation 'com.github.gavlyukovskiy:p6spy-spring-boot-starter:1.9.0'
```